

7.7.3 Hands-On Practice

PROJECT 1: Book CRM

DIFFICULTY LEVEL: Intermediate

Overview

Use flexbox and media queries to create a responsive layout as shown in Figure 7.47. Add in some simple transition and filter effects.

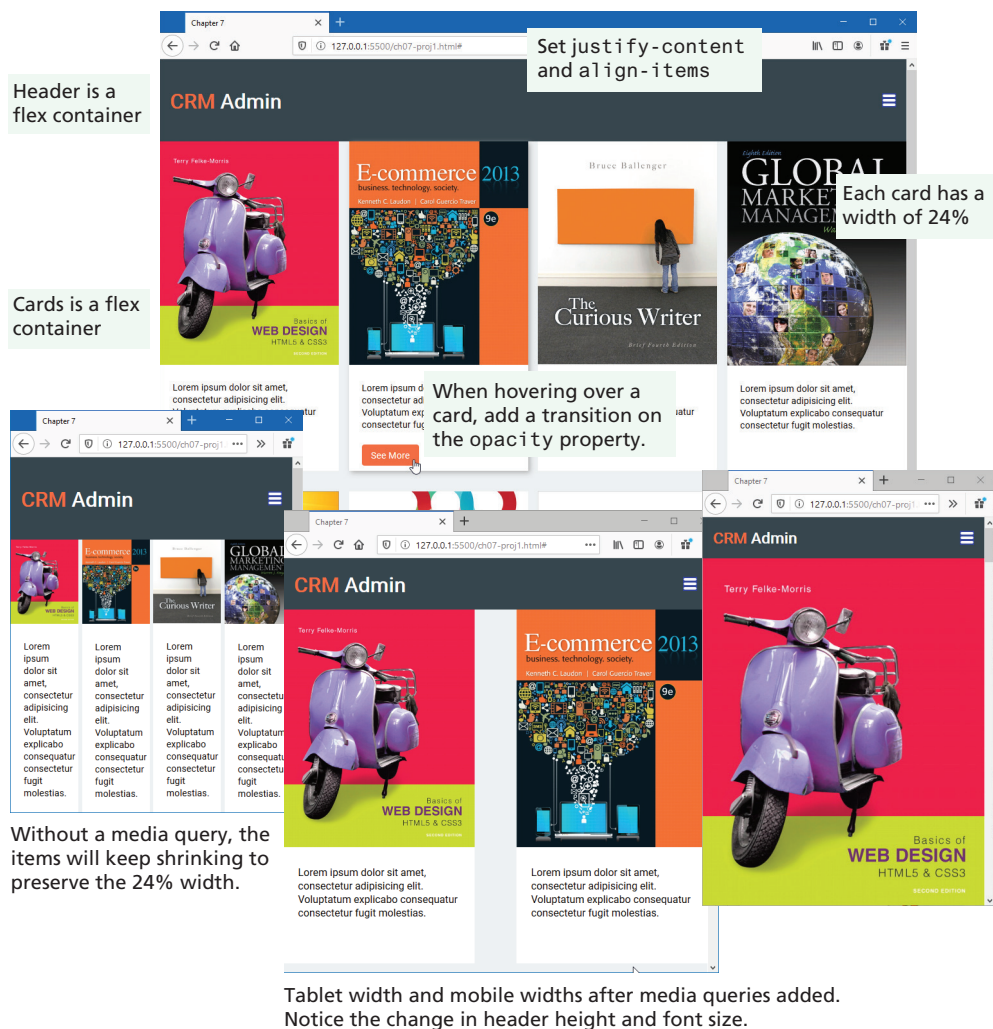


FIGURE 7.47 Completed Project 1

Instructions

1. Examine **ch07-proj1.html** in the browser. You will be modifying the CSS only.
2. Begin modifying **ch07-proj1.css** by using flex layout for the `<header>` element. Set the `justify-content` property of the `<header>` so that its content (the `<h1>` and the `<img>`) spaces itself out to the left and right edges of its flex container. To vertically align the heading and image within the header, set the `align-items` property as well. See Figure 7.13 for guidance.
3. Right now each card fills the entire width of the available space. Change the width of the card class to 24%. By taking less than a quarter of the available space, we will eventually be able to fit four cards on a row.
4. Use flexbox mode for the `container` class. Set its `align-items` and `justify-content` properties to `center`.
5. Use flexbox mode for the `cards` class. Set its `justify-content` property so that its items space themselves out to the left and right edges of its flex container just as you did with the header. When you test this step, examine the book images when you shrink the browser width: notice how they extend beyond the card width.
6. Set the `max-width` property of the card images to 100%. This will ensure these images scale themselves down to fit available space in their container. Test by shrinking the browser width.
7. Add a saturation filter of 150% when hovering over any of the book covers. Add a drop shadow (use `box-shadow`) when hovering over the card. Set the initial `opacity` of the button in the card to 0. Set the opacity of the button to 1 when the user hovers anywhere over the card; in addition, add a 1 second transition on the opacity property.
8. When you resize the browser, the flex containers will continue to shrink in order to maintain four columns because you set the width to 24% in step 3. Add a media query for mobile-sized screens (below 480px) and for tablet-sized screens (between 481px and 768px). For mobile screens, set the card width to 100% and for tablet screens, set the width to 45%.
9. Finally, fine tune the tablet and mobile settings by increasing the `--card-font-size` variable to 100% within the media query for those two widths. For mobile portrait, also shrink the size of the header by changing the `--header-font-size` variable to 24px and the `--header-height` property to 4em. For tablet, set the `--header-height` property to 5em.

Guidance and Testing

1. Break this problem down into smaller steps. Begin first with header, then the cards, then add in the media queries. For each of the instruction steps, test each change you make.
2. Take another look at Figure 7.12. A flexbox container positions items within its container along the major axis. To modify how the container positions items within the container, you use the properties shown in Figure 7.13.

3. Be sure to test your page at different browser widths. To test mobile widths, you may need to make use of the browser's device toolbar.

PROJECT 2: Difficulty Level: Intermediate

DIFFICULTY LEVEL: Intermediate

Overview

Use grid layout mode and media queries to create the complex layout shown in Figure 7.48. Read the Guidance and Testing section first before starting this project.

Instructions

1. Examine `ch07-proj2.html` in the browser.
2. Begin modifying `ch07-proj2.css` by setting up the grid so that it appears similar to that shown in Figure 7.48. Notice that it uses a 10-column grid with a constant grid size of `1fr`. It has four rows with the heights indicated in the figure. The grid gap is `25px`. Some of the cells span multiple columns or multiple rows. You can achieve this using grid areas or using `span` with the `grid-row` and `grid-column` properties. If using the later approach, you can specify the grid position for each individual cell, or set up styles for spanning rows and columns and then assign them to the cells as needed, as shown in the following:

```
.widthDouble { grid-column: span 2; }
...
<div class="widthDouble" id="a">
```

3. Now you need to set the styling for the grid cells (i.e., the top-level `<div>` elements within the container). Grid cells B, C, E, F, H, L, M, O, P, and Q have `background-images` set via CSS. Cells D, I, J, and K have background colors: two with buttons and two with icons from the font-awesome icon library. Each cell should use flex layout for its contents.
4. The images within cells A, G, and N are not set via `background-image` but are `<img>` elements. The other text will need to be added to the different elements in the HTML.
5. Add media queries for tablet widths and for mobile widths. You can use `850px` and `1100px` as your device settings. For tablet and mobile widths, reduce the font size of your text. For mobile widths only, switch to single column and multiple rows (thus each grid row will contain a single cell); each row can have the same height as the non-mobile version, or you can create custom heights for each row.

Guidance and Testing

1. Break this problem down into smaller steps. Begin by constructing the grid structure, as shown in the first screen capture in Figure 7.48 (and described in